

Week 6 Lifetime

Chapter 6: Classes Deep Dive - Constructor, Destructor, Inheritance, Object Lifetime

📁 Basic File Management Rules for Beginners

The Golden Rule: Keep your projects organized from day one. Messy files = messy code.

Do's:

- **Create a dedicated folder** for every project (e.g., `C:\Projects\` or `~/Documents/Code/`)
- **Use clear folder names:** `MyFirstWebsite`, `Python-Homework`, `2026-Learn-JavaScript`
- **Separate by file type:** Put images in `images/`, documents in `docs/`, source code in `src/`
- **Date your archives:** If you need backups, use `ProjectName-v1`, `ProjectName-v2` or add dates `2026-04-01-backup`

Don'ts:

- ❌ **Never save source files to Desktop** — it becomes a chaotic mess within a week
- ❌ Don't use spaces or Chinese in folder/file names (use `MyProject` or `my_project`, not `My Project`, not `程设`)
- ❌ Don't dump everything in one folder — you'll never find anything later

Quick Example Structure:

Code block

```
1  Documents/
2  └── Code/
3      ├── Learn-C++/
4          ├── week1
5              ├── main.cpp
6              ├── student.cpp
7              ├── student.h
8          └── week2
```

Why this matters: Clean folders help you find bugs faster, prevent accidental deletion, and teach professional habits that employers expect.

Pro tip: Treat your computer like a workspace, not a junk drawer.

0. Bilingual Quick Reference

Table 1: Core Concepts

Chinese (中文)	English	Description
类	Class	A user-defined type that bundles data and behavior.
对象	Object	An instance of a class stored in memory.
构造函数	Constructor	A special function called when an object is created.
析构函数	Destructor	A special function called when an object is destroyed.
继承	Inheritance	Building a new class from an existing class.
封装	Encapsulation	Hide internals, expose safe public operations.

生存期 / 生命周期	Lifetime	Duration from variable/object creation to destruction.
作用域	Scope	Region of code where a variable name is visible/accessible.
局部变量	Local Variable	Variable defined inside a function/block; destroyed when leaving scope.
全局变量	Global Variable	Variable defined outside any function; lives for entire program.
静态变量	Static Variable	Variable with extended lifetime but restricted scope.
初始化	Initialization	Giving a variable its initial value at creation time.
初始化列表	Initialization List	Constructor syntax for initializing members before body executes.
公有继承	Public Inheritance	<code>public</code> base members keep access level in derived class.
私有继承	Private Inheritance	Base <code>public</code> / <code>protected</code> members become <code>private</code> in derived.
对象数组	Object Array	Array where each element is an object of a class.
基类	Base Class	Parent class being inherited from.
派生类	Derived Class	Child class inheriting from base class.

Table 2: Symbols & Keywords

Symbol/Keyword	English Name	Chinese (中文)	Usage/Meaning
::	Scope Resolution	作用域解析符	Define member functions outside class
.	Member Access	成员访问符	Access member via object
->	Arrow Operator	箭头操作符	Access member via pointer
:	Inheritance/Init List	继承/初始化列表符号	class B : public A , constructor init list
const	Const Qualifier	常量限定符	Promise not to modify object state
static	Static Keyword	静态关键字	Extend variable lifetime, limit scope
this	This Pointer	自身指针	Points to current object instance
private	Private Access	私有访问权限	Only accessible within same class
protected	Protected Access	保护访问权限	Accessible within class and derived classes
public	Public Access	公有访问权限	Accessible from anywhere

Table 3: Lifetime-Related Terms

English	Chinese (中文)	Explanation
Automatic Storage Duration	自动存储期	Local variables; created on entry, destroyed on exit

		from scope.
Static Storage Duration	静态存储期	Global and static variables; created at program start, destroyed at program end.
Dynamic Storage Duration	动态存储期	Variables created with <code>new</code> ; programmer controls lifetime.
Scope	作用域	Where the variable name can be used to refer to the entity.
Visibility	可见性	Whether a name can be referenced from a given point in code.
Object Lifetime	对象生命周期	Duration from construction to destruction.
RAII	资源获取即初始化	Resource lifecycle is bound to object lifetime.

Term intuition (beginner-friendly, Chinese)

Use this as a **mental hook**; formal definitions are in the tables above.

- **类 / Class**: 把数据和行为绑在一起；若只用零散全局变量，很难保证状态一致。
- **构造函数**: 对象"出生"时自动执行，用来把对象放到合法状态；反例：先创建对象再漏写字段赋值。
- **析构函数**: 对象"销毁"前自动执行；反例：在析构里写复杂业务逻辑，难以调试。
- **继承**: 复用并扩展已有类型；反例：仅为复制代码而继承 unrelated 类。
- **生存期 / Lifetime**: 变量从"诞生"到"死亡"的时间段；局部变量随代码块生灭，全局变量随程序生灭。
- **作用域 / Scope**: 变量名在哪些代码行可以被使用；出了作用域，变量名就"看不见"了（但对象可能还活着，如果有其他引用）。
- **局部变量**: 函数内部的变量，离开函数就销毁；反例：需要长期保存的数据用了局部变量。

- **全局变量:** 所有函数都能访问的变量，程序结束才销毁；反例：滥用全局变量导致代码难以理解和测试。
- **静态变量:** 只在定义处初始化一次，值会保持到下次访问；反例：在多线程环境不注意同步问题。
- **初始化列表:** 在构造函数体执行前就初始化成员，更高效且必需用于 const 和引用成员。

1. Variable Lifetime and Scope Fundamentals (变量生存期与作用域基础)

1.1 The Core Concept: Why Lifetime Matters

Every variable in C++ has a **lifetime** (生存期) — the period during which it exists in memory and holds a valid value. Understanding lifetime is crucial because:

1. **Accessing destroyed variables causes bugs:** Using a variable after its lifetime ends is undefined behavior.
2. **Resource management:** Files, network connections, and memory should be released when no longer needed.
3. **Program correctness:** Knowing when objects are created and destroyed helps predict program behavior.

1.2 Three Fundamental Storage Durations

C++ has three main storage duration categories:

Storage Duration	Chinese	Typical Variables	Creation Time	Destruction Time
Automatic	自动存储期	Local variables inside functions/blocks	When entering scope	When leaving scope

Static	静态存储期	Global variables, <code>static</code> local variables	Program start	Program end
Dynamic	动态存储期	Objects created with <code>new</code>	When <code>new</code> is called	When <code>delete</code> is called

1.3 Local Variables (Automatic Storage)

Local variables are created when execution enters their scope and destroyed when leaving.

Code block

```
1  void demonstrateLocalVariables() {
2      int a = 10;           // Created here
3      cout << a << endl;    // Valid: a exists
4
5      if (a > 5) {
6          int b = 20;        // b created when entering if-block
7          cout << b << endl;
8      }                     // b destroyed here
9
10     // cout << b << endl; // ERROR: b no longer exists!
11 }                           // a destroyed here
12
13 int main() {
14     demonstrateLocalVariables();
15     // All local variables from the function are now destroyed
16     return 0;
17 }
```

Key Characteristics of Local Variables:

- Created on the **stack** (typically)

- Automatically destroyed when scope ends
- Each function call gets fresh copies
- Safe and easy to use — preferred for most temporary data

1.4 Global Variables (Static Storage)

Global variables are defined outside all functions and exist for the entire program duration.

Code block

```
1  #include <iostream>
2  using namespace std;
3
4  // Global variables - exist for entire program
5  int globalCounter = 0;
6  string systemName = "University System";
7
8  void incrementCounter() {
9      globalCounter++;    // Accessible from any function
10     cout << "Counter: " << globalCounter << endl;
11 }
12
13 void printSystemInfo() {
14     cout << "System: " << systemName << endl;
15 }
16
17 int main() {
18     incrementCounter();    // Prints: Counter: 1
19     incrementCounter();    // Prints: Counter: 2
20     printSystemInfo();    // Prints: System: University System
21     return 0;
22 }
```

When to Use Global Variables (Sparingly!):

- Truly global configuration (like `systemName` above)
- Constants that never change (use `const`)
- Singleton-like shared resources

Dangers of Global Variables:

- Any function can modify them — hard to track changes
- Makes testing difficult (functions have hidden dependencies)
- Name collisions in large programs

- Initialization order issues across files

Better Practice for Global Constants:

Code block

```
1  // In header file
2  namespace Config {
3      const int MAX_COURSES = 50;           // Better: namespace + const
4      const double DEFAULT_GPA = 0.0;
5  }
6
7  Config::MAX_COURSES
```

1.5 Static Variables (The Best of Both Worlds)

Static variables combine **extended lifetime** (like globals) with **restricted scope** (like locals).

1.5.1 Static Local Variables

Static local variables maintain their value between function calls but are only accessible within the function.

Code block

```
1  void visitCount() {
2      static int visitCount = 0; // Initialized ONLY once!
3      visitCount++;
4      cout << "Visit #" << visitCount << endl;
5  }
6
7  int main() {
8      visitCount(); // Visit #1
9      visitCount(); // Visit #2
10     visitCount(); // Visit #3
11     // visitCount is not accessible here (good! scope is limited)
12     return 0;
13 }
```

Use Cases for Static Local Variables:

- Function call counters
- Caching/memoization
- Singleton pattern implementation
- Any data that needs to persist but shouldn't be globally visible

1.5.2 Static Member Variables (Class-Level Data)

Static member variables are shared across all objects of a class.

Code block

```
1  class Student {
2  private:
3      int id;
4      string name;
5      static int totalStudents; // Shared by ALL students
6
7  public:
8      Student(int i, const string& n) : id(i), name(n) {
9          totalStudents++;
10         cout << "Student created. Total: " << totalStudents << endl;
11     }
12
13     ~Student() {
14         totalStudents--;
15         cout << "Student destroyed. Total: " << totalStudents << endl;
16     }
17
18     static int getTotalStudents() {
19         return totalStudents;
20     }
21 };
22
23 // REQUIRED: Define static member outside class
24 int Student::totalStudents = 0;
25
26 int main() {
27     cout << "Initial total: " << Student::getTotalStudents() << endl;
28
29     Student s1(1001, "Alice"); // Total: 1
```

```

30     Student s2(1002, "Bob");    // Total: 2
31
32     {
33         Student s3(1003, "Carol"); // Total: 3
34     } // s3 destroyed, Total: 2
35
36     cout << "Final total: " << Student::getTotalStudents() << endl; // 2
37     return 0;
38 }

```

Key Points about Static Members:

- Only ONE copy exists, shared by all objects
- Must be defined outside the class (in .cpp file)
- Can be accessed via class name: `Student::getTotalStudents()`
- Useful for: counting objects, shared configuration, class-wide constants

1.6 Scope vs Lifetime (重要区别)

Scope (作用域): Where the variable name is visible — compile-time concept

Lifetime (生存期): When the variable actually exists in memory — runtime concept

They usually match, but not always:

Code block

```

1  void scopeVsLifetimeDemo() {
2      // Example: static variable has limited scope but extended lifetime
3      static int counter = 0;
4      counter++;
5
6      // 'counter' is only visible inside this function (scope)
7      // But 'counter' exists for entire program (lifetime)
8  }
9
10 // Example: dynamically allocated object
11 void dynamicDemo() {
12     // Local pointer - automatic storage
13     int* ptr = new int(42); // Dynamic int created

```

```

14
15     // ptr itself is local (will be destroyed when function ends)
16     // But the int it points to lives until delete is called!
17
18     delete ptr; // Now the dynamic int is destroyed
19 } // ptr destroyed here, but if we forgot delete, memory leaks!

```

Checkpoint 1 (you should now be able to)

1. Explain the difference between scope and lifetime with your own example.
2. Predict how many times `static int count = 0;` is initialized in a function called 5 times.
3. Name one danger of global variables and one appropriate use case.
4. When does a local variable's lifetime end? When does a static variable's lifetime end?

2. Constructors and Initialization Deep Dive (构造函数与初始化深入)

2.1 The Problem: Why Constructors Matter for Lifetime

Proper initialization is the **start of an object's lifetime**. A poorly initialized object is a bug waiting to happen.

Before Constructors (Procedural Style):

Code block

```

1 // Old withProf style - risky!
2 Professor prof;
3 prof.id = 1001; // What if we forget this line?
4 prof.name = "Dr. Smith"; // What if name stays empty?
5 prof.courseCount = 0; // Manual initialization everywhere
6 // Object could be in invalid state between creation and initialization!

```

After Constructors (Object-Oriented Style):

Code block

```
1 // Better: Object is valid immediately after birth
2 Professor prof(1001, "Dr. Smith"); // All fields initialized at once
3 // No possibility of half-initialized object!
```

2.2 Constructor Types and When They're Called

Code block

```
1 class Professor {
2     private:
3         int id;
4         string name;
5         string courses[50];
6         int courseCount;
7
8     public:
9         // 1. Default Constructor - called when no arguments provided
10        Professor() : id(0), courseCount(0) {
11            for (int i = 0; i < 50; i++) {
12                courses[i] = "";
13            }
14        }
15
16        // 2. Parameterized Constructor - called with arguments
17        Professor(int id, const string& name)
18            : id(id), name(name), courseCount(0) {
19            for (int i = 0; i < 50; i++) {
20                courses[i] = "";
21            }
22        }
23
24        // 3. Copy Constructor - called when copying objects
25        Professor(const Professor& other)
26            : id(other.id), name(other.name), courseCount(other.courseCount) {
27            for (int i = 0; i < courseCount; i++) {
28                courses[i] = other.courses[i];
29            }
26        }
```

```

30         for (int i = courseCount; i < 50; i++) {
31             courses[i] = "";
32         }
33     }
34 };

```

Constructor Call Examples:

Code block

```

1  Professor p1;                                // Default constructor
2  Professor p2(1001, "Dr. Smith");              // Parameterized constructor
3  Professor p3 = p2;                            // Copy constructor
4  Professor p4(p2);                            // Also copy constructor
5  void func(Professor p);
6  func(p2);    // Copy constructor (pass by value)
7  Professor returnProf()
8  {
9      Professor p;
10     return p;
11 } // Copy/move constructor
12
13 p2 = returnProf();

```

2.3 Member Initialization List (成员初始化列表)

The initialization list runs **before** the constructor body executes. It is the preferred way to initialize members.

Syntax:

Code block

```

1  ClassName(parameters) : member1(value1), member2(value2), ... {
2      // Constructor body
3  }

```

Why Prefer Initialization Lists:

Aspect	Initialization List	Assignment in Body
Timing	Before body executes	During body execution
Efficiency	Direct initialization	Default construction + assignment
const members	REQUIRED	Not possible
Reference members	REQUIRED	Not possible
Base class initialization	REQUIRED	Not possible

Example Comparison:

Code block

```
1  // Less efficient: default construct then assign
2  class Professor {
3      string name;
4      int id;
5  public:
6      Professor(int i, const string& n) {
7          name = n;    // First: default constructs empty string
8          id = i;      // Then: assigns new value
9      }
10 };
11
12 // More efficient: direct initialization
13 class Professor {
14     string name;
15     int id;
16 public:
17     Professor(int i, const string& n)
18         : name(n)    // Direct construct with value
19         , id(i)      // Direct initialization
20     {
21         // Empty body is fine!
22     }
23 };
```

2.4 Initialization Order (Important!)

Members are initialized in the **order they are declared in the class**, NOT the order in the initialization list!

Code block

```
1  class InitializationOrderDemo {
2      int a; // Declared first
3      int b; // Declared second
4      int c; // Declared third
5
6  public:
7      // Even though list order is c, a, b...
8      InitializationOrderDemo()
9          : c(3) // ...a is initialized first (declared first!)
10         , a(1) // ...then b (declared second!)
11         , b(2) // ...then c (declared third!)
12     {
13         cout << a << b << c << endl; // Prints: 1 2 3
14     }
15 };
```

Best Practice: Always write initialization list in the same order as member declarations to avoid confusion.

2.5 Constructor Validation: Enforcing Invariants

Use constructors to ensure objects start in valid states.

Code block

```
1  class Professor {
2  private:
```

```

3      int id;
4      string name;
5      int courseCount;
6      static const int MAX_COURSES = 50;
7
8  public:
9      Professor(int id, const string& name)
10         : id(id > 0 ? id : -1)           // Validate: ID must be positive
11         , name(name.empty() ? "Unknown" : name) // Validate: name can't be
empty
12         , courseCount(0)                 // Always start with 0 courses
13     {
14         // Post-construction validation
15         if (this->id == -1) {
16             cout << "Warning: Invalid ID provided, using -1\n";
17         }
18     }
19
20     bool isValid() const { return id > 0; }
21 };

```

2.6 Default Member Initializers (C++11 and later)

You can provide default values directly in the class definition:

Code block

```

1  class ModernProfessor {
2  private:
3      int id = 0;           // Default member initializer
4      string name = "Unknown";
5      int courseCount = 0;
6      static const int MAX_COURSES = 50;
7
8  public:
9      // Default constructor - members already initialized
10     ModernProfessor() = default;
11
12     // Parameterized constructor - can override defaults
13     ModernProfessor(int i, const string& n)
14         : id(i > 0 ? i : 0)
15         , name(n.empty() ? "Unknown" : n)

```

```

16     {
17         // courseCount already 0 from default initializer
18     }
19 };

```

2.7 Common Constructor Mistakes

Mistake	Why Wrong	Correct Way
<code>id = id;</code>	Parameter shadows member, self-assigns	<code>this->id = id</code> or init list
Forgetting to initialize all members	Object in undefined state	Use init list, provide defaults
Complex logic in constructor	Can throw exceptions, hard to debug	Keep simple, validate inputs
Calling virtual functions	Derived class not yet constructed	Avoid virtual calls in constructor
<code>Professor()</code> instead of <code>Professor</code>	Creates temporary, not variable	Just write <code>Professor p;</code>

Checkpoint 2 (you should now be able to)

- Write a constructor using initialization list for a `Course` class with `code`, `name`, and `capacity`.
- Explain why `const` members must use initialization lists.
- Identify the bug: `Professor(int id) { id = id; }`
- Predict initialization order given member declaration order vs init list order.

3. Destructors and Object Lifetime End (析构函数与对象生存期结束)

3.1 Destructor Fundamentals

A destructor is called automatically when an object's lifetime ends. Its job is to clean up resources and perform final actions.

Syntax:

Code block

```
1  class ClassName {
2  public:
3      ~ClassName() { // Destructor: ~ + ClassName
4          // Cleanup code
5      }
6  };
```

Characteristics:

- No return type, no parameters
- Exactly one destructor per class
- Cannot be overloaded
- Automatically generated if not defined

3.2 When Is the Destructor Called?

Code block

```
1  void demonstrateDestructorTiming() {
2      cout << "Entering function\n";
3
4      Professor p1(1001, "Dr. A"); // Constructor called
5      cout << "p1 created\n";
6
7      { // New scope
8          Professor p2(1002, "Dr. B"); // Constructor
9          cout << "p2 created\n";
10     } // p2's destructor called here!
11 }
```

```

12     cout << "p2 destroyed\n";
13     cout << "Leaving function\n";
14 } // p1's destructor called here!
15
16 int main() {
17     demonstrateDestructorTiming();
18     cout << "Function returned\n";
19     return 0;
20 }

```

Output:

Code block

```

1  Entering function
2  p1 created
3  p2 created
4  p2 destroyed
5  Leaving function
6  p1 destroyed
7  Function returned

```

3.3 Destructor Order: Reverse of Construction

When multiple objects are destroyed, they follow **reverse construction order** (LIFO: Last In, First Out).

Code block

```

1  class Trace {
2      string label;
3  public:
4      Trace(const string& s) : label(s) {
5          cout << "Creating: " << label << endl;
6      }
7      ~Trace() {
8          cout << "Destroying: " << label << endl;
9      }
10 };

```

```

11
12 void nestedScopes() {
13     Trace a("outer");
14     {
15         Trace b("inner-1");
16         Trace c("inner-2");
17     }
18     Trace d("outer-end");
19 }

```

Output:

Code block

```

1  Creating: outer
2  Creating: inner-1
3  Creating: inner-2
4  Destroying: inner-2
5  Destroying: inner-1
6  Creating: outer-end
7  Destroying: outer-end
8  Destroying: outer

```

3.4 Practical Example: Session Tracking

Code block

```

1  class AuditSession {
2      string sessionId;
3      static int activeSessions;
4
5  public:
6      AuditSession(const string& id) : sessionId(id) {
7          activeSessions++;
8          cout << "[" << sessionId << "]" Session started. "
9              << "Active: " << activeSessions << endl;
10     }
11
12     ~AuditSession() {
13         activeSessions--;

```

```

14         cout << "[" << sessionId << "]" Session ended. "
15             << "Active: " << activeSessions << endl;
16     }
17
18     static int getActiveCount() { return activeSessions; }
19 };
20
21 int AuditSession::activeSessions = 0;
22
23 void processProfessor() {
24     AuditSession session("prof-1001");
25     // Do work...
26     cout << "Processing professor...\n";
27 } // session destroyed here
28
29 void processCourse() {
30     AuditSession session("course-CS101");
31     processProfessor(); // Nested session
32     cout << "Processing course...\n";
33 } // course session destroyed
34
35 int main() {
36     cout << "Starting main\n";
37     processCourse();
38     cout << "Main ending. Active sessions: "
39         << AuditSession::getActiveCount() << endl;
40     return 0;
41 }

```

Output:

Code block

```

1 Starting main
2 [course-CS101] Session started. Active: 1
3 [prof-1001] Session started. Active: 2
4 Processing professor...
5 [prof-1001] Session ended. Active: 1
6 Processing course...
7 [course-CS101] Session ended. Active: 0
8 Main ending. Active sessions: 0

```

3.5 Object Arrays and Destructors

When an array of objects is destroyed, each element's destructor is called in **reverse order**.

Code block

```
1  class Simple {
2      int id;
3  public:
4      Simple(int i) : id(i) { cout << "Created #" << id << endl; }
5      ~Simple() { cout << "Destroyed #" << id << endl; }
6  };
7
8  int main() {
9      Simple arr[3] = {Simple(1), Simple(2), Simple(3)};
10     cout << "Array created\n";
11     // arr destroyed when main ends
12     // Destruction order: 3, 2, 1 (reverse)
13     return 0;
14 }
```

Output:

Code block

```
1  Created #1
2  Created #2
3  Created #3
4  Array created
5  Destroyed #3
6  Destroyed #2
7  Destroyed #1
```

3.6 Common Destructor Mistakes

Mistake	Symptom	Fix

Throwing exceptions	Program termination	Never throw from destructor
Heavy business logic	Hard-to-debug side effects	Keep cleanup only
Forgetting virtual destructor	Memory leaks in inheritance	Add <code>virtual ~Base()</code> for base classes
Accessing other objects	May already be destroyed	Don't depend on external state

Checkpoint 3 (you should now be able to)

1. Predict destructor call order for three nested objects.
2. Explain why destructors run in reverse order of constructors.
3. Write a simple class that tracks how many instances are active.
4. When is a local object's destructor called? A global object's destructor?

4. Object Arrays (对象数组)

4.1 Declaring and Initializing Object Arrays

Object arrays allow you to manage collections of objects with the same lifetime.

Code block

```

1  class Professor {
2      int id;
3      string name;
4  public:
5      Professor() : id(0), name("Unknown") {} // Default constructor required!
6      Professor(int i, const string& n) : id(i), name(n) {}
7      void print() const { cout << id << ": " << name << endl; }
8  };

```

```

9
10 int main() {
11     // Array of 3 Professors - default constructor called for each
12     Professor faculty[3];
13
14     // Each element can be accessed and modified
15     faculty[0] = Professor(1001, "Dr. Smith");
16     faculty[1] = Professor(1002, "Dr. Jones");
17     faculty[2] = Professor(1003, "Dr. Lee");
18
19     for (int i = 0; i < 3; i++) {
20         faculty[i].print();
21     }
22
23     return 0; // All 3 destructors called in reverse order
24 }

```

4.2 Initialization List for Arrays

You can initialize arrays using brace-enclosed lists:

Code block

```

1 // Explicit initialization for each element
2 Professor faculty[3] = {
3     Professor(1001, "Dr. Smith"),
4     Professor(1002, "Dr. Jones"),
5     Professor(1003, "Dr. Lee")
6 };
7
8 // With C++11 and later, can be simpler:
9 Professor faculty2[3] = {
10     {1001, "Dr. Smith"}, // Implicit constructor call
11     {1002, "Dr. Jones"},
12     {1003, "Dr. Lee"}
13 };

```

4.3 Arrays in Classes: Managing Collections

A common pattern is for a class to manage an array of objects:

Code block

```
1  class Department {
2  private:
3      string name;
4      Professor professors[10]; // Fixed-size array
5      int professorCount;
6
7  public:
8      Department(const string& n)
9          : name(n), professorCount(0) {
10         // professors array already default-constructed
11     }
12
13     bool addProfessor(int id, const string& profName) {
14         if (professorCount >= 10) {
15             return false; // Department is full
16         }
17         professors[professorCount] = Professor(id, profName);
18         professorCount++;
19         return true;
20     }
21
22     void printRoster() const {
23         cout << "Department: " << name << endl;
24         cout << "Professors (" << professorCount << "):\n";
25         for (int i = 0; i < professorCount; i++) {
26             cout << "  ";
27             professors[i].print();
28         }
29     }
30 };
```

4.4 Lifetime of Array Elements

All elements in an array share the **same lifetime** as the array itself:

```

1 void arrayLifetimeDemo() {
2     // All 5 Professors created here
3     Professor team[5];
4     cout << "Team assembled\n";
5
6     // ... use the team ...
7
8 } // All 5 destructors called here (in reverse order: 4, 3, 2, 1, 0)
9
10 int main() {
11     arrayLifetimeDemo();
12     cout << "Team disbanded\n";
13     return 0;
14 }

```

4.5 Static Arrays vs Local Arrays

Code block

```

1 // Global array - static storage duration
2 Professor globalFaculty[5]; // Lives entire program, initialized before main()
3
4 void functionWithLocalArray() {
5     // Local array - automatic storage duration
6     Professor localTeam[3]; // Created on entry, destroyed on exit
7
8     // Static local array - static storage duration, local scope
9     static Professor staticTeam[2]; // Created once, persists between calls
10 }

```

4.6 Common Array Mistakes

Mistake	Why Wrong	Correct Way
Missing default constructor	Array initialization fails	Provide default constructor
Array bounds violation		Always check index < size

	Undefined behavior, crashes	
<pre>Professor arr[3] = Professor(1, "A");</pre>	Tries to assign one to array	Use brace list or loop
Returning local array	Dangling reference	Use static, dynamic allocation, or pass in buffer

Checkpoint 4 (you should now be able to)

1. Create an array of 5 `Course` objects and initialize them with different values.
2. Explain when array element destructors are called.
3. Why does `Professor arr[10];` require a default constructor?
4. Write a `University` class that manages an array of `Department` objects.

5. Access Control and Encapsulation (访问控制与封装)

5.1 The Three Access Levels

Access specifiers control **who can use** class members:

Code block

```

1  class AccessDemo {
2  private:           // Only this class can access
3      int secretId;
4      void internalHelper() {}
5
6  protected:       // This class and derived classes can access
7      int familyId;
8      void familyMethod() {}
9
10 public:           // Anyone can access

```

```

11     int publicId;
12     void publicMethod() {}
13 };

```

Access Level	Same Class	Derived Class	Outside Code
private	✓	✗	✗
protected	✓	✓	✗
public	✓	✓	✓

5.2 Encapsulation in Practice

Goal: Hide internal state, expose controlled interface.

Code block

```

1  class Professor {
2  private:
3      int id;                // Private: can't be modified directly
4      string name;
5      string courses[50];
6      int courseCount;
7      static const int MAX_COURSES = 50;
8
9  public:
10     // Constructor - controlled initialization
11     Professor(int id, const string& name);
12
13     // Getter methods (read-only access)
14     int getId() const { return id; }
15     string getName() const { return name; }
16     int getCourseCount() const { return courseCount; }
17
18     // Controlled modification
19     void addCourse(const string& course);
20     void removeCourse(const string& course);
21

```

```

22     // Utility method using internal state
23     void print() const;
24     bool isTeaching(const string& course) const;
25 };

```

Benefits of Encapsulation:

1. **Invariant protection:** `id` can't become negative because we validate in constructor
2. **Flexibility:** Can change internal representation without affecting users
3. **Debugging:** All modifications go through controlled methods
4. **Safety:** Can't accidentally corrupt object state

5.3 Friend Functions (Occasional Exception)

Sometimes you need external functions to access private members:

Code block

```

1  class Professor {
2  private:
3      int id;
4      string name;
5
6  public:
7      Professor(int i, const string& n) : id(i), name(n) {}
8
9      // Grant access to specific external function
10     friend bool haveSameDepartment(const Professor& a, const Professor& b);
11 };
12
13 // Friend function can access private members
14 bool haveSameDepartment(const Professor& a, const Professor& b) {
15     // Hypothetically checking if same department
16     // Can access a.id and b.id directly
17     return (a.id / 1000) == (b.id / 1000); // Same department prefix
18 }

```

Use friends sparingly — they break encapsulation. Prefer public interfaces.

Checkpoint 5 (you should now be able to)

1. List the three access levels and who can access each.
2. Explain why `private` members with `public` getters are better than public members.
3. Write a `Course` class with private `code` and `capacity`, public getters, and controlled enrollment method.

6. Inheritance with Constructors and Destructors (继承中的构造与析构)

6.1 Basic Inheritance Syntax

Inheritance allows a derived class to reuse and extend a base class:

Code block

```
1  // Base class
2  class Person {
3  protected: // Accessible to derived classes
4      int id;
5      string name;
6
7  public:
8      Person(int i, const string& n) : id(i), name(n) {}
9
10     void printBasic() const {
11         cout << "ID: " << id << ", Name: " << name << endl;
12     }
13
14     int getId() const { return id; }
15     string getName() const { return name; }
16 };
17
```

```

18 // Derived class
19 class Student : public Person {
20 private:
21     double gpa;
22     string major;
23
24 public:
25     // Must call base constructor in initialization list
26     Student(int i, const string& n, double g, const string& m)
27         : Person(i, n)      // Base constructor first!
28         , gpa(g)
29         , major(m)
30     {}
31
32     void print() const {
33         printBasic(); // Reuse base method
34         cout << "GPA: " << gpa << ", Major: " << major << endl;
35     }
36 };
37
38 // Another derived class
39 class Professor : public Person {
40 private:
41     string department;
42     int yearsOfService;
43
44 public:
45     Professor(int i, const string& n, const string& dept, int years)
46         : Person(i, n)
47         , department(dept)
48         , yearsOfService(years)
49     {}
50
51     void print() const {
52         printBasic();
53         cout << "Department: " << department
54             << ", Experience: " << yearsOfService << " years" << endl;
55     }
56 };

```

6.2 Construction Order in Inheritance

When creating a derived object:

1. **Base class** constructor runs first
2. **Member variables** are initialized (in declaration order)
3. **Derived class** constructor body runs

Code block

```
1  class Base {
2  public:
3      Base() { cout << "Base constructor\n"; }
4  };
5
6  class Member {
7  public:
8      Member() { cout << "Member constructor\n"; }
9  };
10
11 class Derived : public Base {
12     Member m;
13 public:
14     Derived() { cout << "Derived constructor\n"; }
15 };
16
17 // Creating Derived:
18 // 1. "Base constructor"
19 // 2. "Member constructor"
20 // 3. "Derived constructor"
```

6.3 Destruction Order in Inheritance

Destruction is the **reverse** of construction:

1. **Derived class** destructor runs
2. **Member variables** are destroyed (reverse declaration order)
3. **Base class** destructor runs

```

1 node class class Base {
2     public:
3         ~Base() { cout << "Base destructor\n"; }
4     };
5
6     class Member {
7     public:
8         ~Member() { cout << "Member destructor\n"; }
9     };
10
11     class Derived : public Base {
12         Member m;
13     public:
14         ~Derived() { cout << "Derived destructor\n"; }
15     };
16
17     // Destroying Derived:
18     // 1. "Derived destructor"
19     // 2. "Member destructor"
20     // 3. "Base destructor"

```

6.4 Access Control in Inheritance

Public Inheritance (`class B : public A`)

- Models **is-a** relationship

- Base `public` members stay `public` in derived
- Base `protected` members stay `protected` in derived

Code block

```

1     class Person {
2     public:
3         void greet() { cout << "Hello!\n"; }
4     };
5
6     class Student : public Person {
7         // greet() is still public here
8     };
9
10    Student s;
11    s.greet(); // OK: Student is-a Person

```

Private Inheritance (`class B : private A`)

- Models **implemented-in-terms-of** relationship

- Base `public` and `protected` members become `private` in derived
- Not an "is-a" relationship to outside world

Code block

```
1  class Logger {
2  protected:
3      void log(const string& msg) { cout << msg << endl; }
4  };
5
6  class CourseManager : private Logger {
7  public:
8      void recordAction(const string& action) {
9          log(action); // Can use internally
10     }
11     // log() is NOT accessible from outside
12 };
13
14 CourseManager cm;
15 // cm.log("test"); // ERROR: log is private!
16 cm.recordAction("Added course"); // OK
```

6.5 Inheritance vs Composition

Use inheritance when:

- Clear "is-a" relationship (Student is-a Person)
- Need to reuse interface, not just implementation
- Derived class should be substitutable for base class

Use composition when:

- "has-a" relationship (Professor has-a array of courses)
- Just need to reuse implementation
- Don't want to expose base class interface

Code block

```
1  // Composition: Professor has-a collection of courses
2  class Professor {
3  private:
4      string courses[50]; // Has courses, not "is" courses
5      int courseCount;
6  };
7
8  // Inheritance: Professor is-a Person
9  class Professor : public Person {
10     // Professor has all Person characteristics
11  };
```

6.6 Protected Members: Balancing Access

Protected members are accessible to derived classes but not outside code:

Code block

```
1  class Person {
2  protected:
3      int id; // Derived classes can access
4      string name;
5
6  private:
7      string ssn; // Only Person can access
8
9  public:
10     Person(int i, const string& n, const string& s)
11         : id(i), name(n), ssn(s) {}
12 };
13
14 class Student : public Person {
15 public:
16     void printId() const {
17         cout << id << endl; // OK: protected
18         // cout << ssn; // ERROR: private!
19     }
20 };
21
22 Student s(1001, "Alice", "123-45-6789");
```

```
23 // cout << s.id; // ERROR: protected, not accessible outside
```

Checkpoint 6 (you should now be able to)

1. Draw the construction/destruction order for a 3-level inheritance chain.
2. Explain the difference between `public` and `private` inheritance.
3. When should you prefer composition over inheritance?
4. Write a `TeachingAssistant` class that inherits from `Student` and adds `teachingHours`.

7. Comprehensive withProf Refactor (综合项目重构)

7.1 Project Requirements

Let's redesign the `withProf` system using all concepts from this chapter:

Requirements:

1. Track professors with IDs, names, and courses
2. Support a department managing multiple professors
3. Use proper initialization and encapsulation
4. Demonstrate object lifetime and arrays

7.2 Class Design

Code block

```
1 // ===== Person.h =====
2 #ifndef PERSON_H
3 #define PERSON_H
4
```

```

5  #include <string>
6  using namespace std;
7
8  class Person {
9  protected:
10     int id;
11     string name;
12
13 public:
14     Person(int id, const string& name);
15     virtual ~Person() = default; // Virtual for proper cleanup in inheritance
16
17     int getId() const { return id; }
18     string getName() const { return name; }
19     void print() const;
20 };
21
22 #endif
23
24 // ===== Person.cpp =====
25 #include "Person.h"
26 #include <iostream>
27
28 Person::Person(int id, const string& name)
29     : id(id > 0 ? id : -1)
30     , name(name.empty() ? "Unknown" : name)
31 {
32     if (this->id == -1) {
33         cout << "Warning: Invalid ID for person\n";
34     }
35 }
36
37 void Person::print() const {
38     cout << "Person [" << id << "]: " << name << endl;
39 }
40
41 // ===== Professor.h =====
42 #ifndef PROFESSOR_H
43 #define PROFESSOR_H
44
45 #include "Person.h"
46 #include <string>
47 using namespace std;
48
49 class Professor : public Person {
50 private:
51     string courses[50];

```

```

52     int courseCount;
53     static const int MAX_COURSES = 50;
54
55 public:
56     Professor();
57     Professor(int id, const string& name);
58
59     void addCourse(const string& courseCode);
60     void removeCourse(const string& courseCode);
61     int getCourseCount() const { return courseCount; }
62     string getCourse(int index) const;
63
64     void print() const; // Overrides Person::print
65 };
66
67 #endif
68
69 // ===== Professor.cpp =====
70 #include "Professor.h"
71 #include <iostream>
72
73 Professor::Professor()
74     : Person(0, "Unknown")
75     , courseCount(0)
76 {
77     for (int i = 0; i < MAX_COURSES; i++) {
78         courses[i] = "";
79     }
80 }
81
82 Professor::Professor(int id, const string& name)
83     : Person(id, name)
84     , courseCount(0)
85 {
86     for (int i = 0; i < MAX_COURSES; i++) {
87         courses[i] = "";
88     }
89 }
90
91 void Professor::addCourse(const string& courseCode) {
92     if (courseCode.empty() || courseCount >= MAX_COURSES) {
93         return;
94     }
95
96     for (int i = 0; i < courseCount; i++) {
97         if (courses[i] == courseCode) {

```

```

98         cout << "Note: " << getName() << " already teaches " << courseCode
    << endl;
99         return;
100     }
101 }
102
103     courses[courseCount] = courseCode;
104     courseCount++;
105 }
106
107 void Professor::removeCourse(const string& courseCode) {
108     for (int i = 0; i < courseCount; i++) {
109         if (courses[i] == courseCode) {
110             for (int j = i; j < courseCount - 1; j++) {
111                 courses[j] = courses[j + 1];
112             }
113             courses[courseCount - 1] = "";
114             courseCount--;
115             return;
116         }
117     }
118 }
119
120 string Professor::getCourse(int index) const {
121     if (index < 0 || index >= courseCount) {
122         return "";
123     }
124     return courses[index];
125 }
126
127 void Professor::print() const {
128     cout << "Professor [" << getId() << "]: " << getName() << endl;
129     cout << " Teaching " << courseCount << " course(s):";
130     for (int i = 0; i < courseCount; i++) {
131         cout << " " << courses[i];
132     }
133     cout << endl;
134 }
135
136 // ===== Department.h =====
137 #ifndef DEPARTMENT_H
138 #define DEPARTMENT_H
139
140 #include "Professor.h"
141 #include <string>
142 using namespace std;
143

```

```

144 class Department {
145 private:
146     string name;
147     Professor faculty[10]; // Fixed-size array of professors
148     int facultyCount;
149     static const int MAX_FACULTY = 10;
150
151 public:
152     Department(const string& name);
153
154     bool addProfessor(int id, const string& name);
155     void assignCourseToProfessor(int profIndex, const string& course);
156
157     void printRoster() const;
158     int getFacultyCount() const { return facultyCount; }
159 };
160
161 #endif
162
163 // ===== Department.cpp =====
164 #include "Department.h"
165 #include <iostream>
166
167 Department::Department(const string& name)
168     : name(name.empty() ? "Unnamed Department" : name)
169     , facultyCount(0)
170 {
171     // faculty array automatically default-constructed
172     cout << "Department '" << this->name << "' created\n";
173 }
174
175 bool Department::addProfessor(int id, const string& name) {
176     if (facultyCount >= MAX_FACULTY) {
177         cout << "Error: Department is at capacity\n";
178         return false;
179     }
180
181     faculty[facultyCount] = Professor(id, name);
182     facultyCount++;
183     cout << "Added professor to department. Total faculty: " << facultyCount
184     << endl;
185     return true;
186 }
187
188 void Department::assignCourseToProfessor(int profIndex, const string& course) {
189     if (profIndex < 0 || profIndex >= facultyCount) {
190         cout << "Error: Invalid professor index\n";

```

```

190         return;
191     }
192     faculty[profIndex].addCourse(course);
193 }
194
195 void Department::printRoster() const {
196     cout << "\n=== " << name << " Roster ===\n";
197     cout << "Faculty count: " << facultyCount << "/" << MAX_FACULTY << endl;
198     for (int i = 0; i < facultyCount; i++) {
199         cout << "  " << (i + 1) << ". ";
200         faculty[i].print();
201     }
202     cout << "=====\n";
203 }
204
205 // ===== main.cpp =====
206 #include "Department.h"
207 #include <iostream>
208 using namespace std;
209
210 // Global department (demonstrates global variable)
211 Department globalCS("Computer Science");
212
213 void demonstrateLocalAndStatic() {
214     cout << "\n--- Local vs Static Demo ---\n";
215
216     // Local professor - automatic storage
217     Professor localProf(2001, "Dr. Local");
218     localProf.addCourse("TEMP101");
219     localProf.print();
220
221     // Static professor - static storage, local scope
222     static Professor staticProf(2002, "Dr. Static");
223     static int callCount = 0; // Static local variable
224     callCount++;
225
226     cout << "Function called " << callCount << " time(s)\n";
227     staticProf.addCourse("STATIC" + to_string(callCount));
228     staticProf.print();
229
230     cout << "--- End of function ---\n";
231     // localProf destroyed here, but staticProf lives on!
232 }
233
234 int main() {
235     cout << "=== University System Demo ===\n";
236

```

```

237 // Using global department
238 globalCS.addProfessor(1001, "Dr. Smith");
239 globalCS.addProfessor(1002, "Dr. Jones");
240 globalCS.assignCourseToProfessor(0, "CS101");
241 globalCS.assignCourseToProfessor(0, "CS202");
242 globalCS.assignCourseToProfessor(1, "MATH101");
243
244 // Demonstrate local vs static
245 demonstrateLocalAndStatic();
246 cout << "\nCalling function again...\n";
247 demonstrateLocalAndStatic();
248
249 // Create local department
250 {
251     cout << "\n--- Creating local department ---\n";
252     Department localMath("Mathematics");
253     localMath.addProfessor(3001, "Dr. LocalMath");
254     localMath.printRoster();
255     cout << "--- Local department going out of scope ---\n";
256 } // localMath and its faculty destroyed here
257
258 // Print final state
259 cout << "\n--- Final Global State ---\n";
260 globalCS.printRoster();
261
262 // Array of professors
263 cout << "\n--- Professor Array Demo ---\n";
264 Professor researchGroup[3] = {
265     Professor(4001, "Researcher A"),
266     Professor(4002, "Researcher B"),
267     Professor(4003, "Researcher C")
268 };
269
270 for (int i = 0; i < 3; i++) {
271     researchGroup[i].addCourse("RESEARCH" + to_string(i + 1));
272     researchGroup[i].print();
273 }
274 // researchGroup destroyed here when main ends
275
276 cout << "\n=== Program Ending ===\n";
277 return 0;
278 // Global objects destroyed here in reverse creation order
279 }

```

7.3 Expected Output and Lifetime Observations

When you run this program, observe:

- 1. **Global department** created before ``main()`` starts
- 2. **Local professor** created and destroyed each function call
- 3. **Static professor** persists between function calls
- 4. **Local department** destroyed when leaving its scope
- 5. **Array elements** destroyed in reverse order
- 6. **Global objects** destroyed after ``main()`` ends

Checkpoint 7 (you should now be able to)

- 1. Explain the lifetime of global vs local vs static variables in this program.
- 2. Trace through when each constructor and destructor is called.
- 3. Modify the code to add a static member counting total professors.
- 4. Create an array of departments instead of a single global one.

8. High-Frequency Mistakes Summary

Mistake	Why Wrong	Correct Way
<code>id = id;</code> in constructor	Self-assignment bug	Use init list or <code>this->id</code>
Forgetting base constructor call	Base part uninitialized	<code>Derived(...) :</code> <code>Base(...), ... {}</code>
Accessing local variable after scope	Undefined behavior, crashes	Don't return pointers/references to locals
	Linker error	

Forgetting <code>static</code> variable initialization		Define <code>Type</code> <code>Class::member = value;</code>
Array bounds violation	Buffer overflow, crashes	Always check <code>index < size</code>
Public data members	Breaks encapsulation	Private members + getters/setters
Missing default constructor for arrays	Compilation error	Provide default constructor
Initialization list order confusion	Hard-to-find bugs	Match declaration order
Confusing scope with lifetime	Logical errors	Scope = visibility, lifetime = existence
Throwing from destructor	Program termination	Never throw exceptions from destructors

9. Practice Exercises

Exercise A: Variable Lifetime Analysis

Given this code, predict the output order of all constructor/destructor messages:

Code block

```
1  class X {
2      int id;
3  public:
4      X(int i) : id(i) { cout << "X" << id << " ctor\n"; }
5      ~X() { cout << "X" << id << " dtor\n"; }
6  };
7
8  X global(1);
9
10 void func() {
```

```

11     X local(2);
12     static X staticLocal(3);
13     X arr[2] = {X(4), X(5)};
14 }
15
16 int main() {
17     func();
18     func();
19     return 0;
20 }

```

Questions:

1. In what order are the constructors called on the first `func()` call?
2. In what order are the destructors called when leaving `func()` the first time?
3. How many times is `X3` constructed? Why?
4. What is the complete output sequence?

Exercise B: Professor Class Enhancement

Enhance the `Professor` class with:

1. Add a static member `totalProfessors` that tracks how many `Professor` objects exist
2. Add a `static int getTotalProfessors()` method
3. Ensure the count is correct when objects are created and destroyed
4. Write a `main()` that demonstrates:
 - Creating professors in different scopes
 - Showing the count changes correctly

Exercise C: Department Array Management

Create a `University` class that:

1. Contains an array of 5 `Department` objects
2. Has methods to add a department at a specific index
3. Has a method to print all departments
4. Demonstrates proper initialization and destruction

Bonus: Add bounds checking and prevent duplicate department names.

Exercise D: Access Control Design

Design a `Course` class with proper encapsulation:

Private members:

- `code` (string): Course code (e.g., "CS101")
- `name` (string): Course name
- `capacity` (int): Maximum enrollment
- `enrolled` (int): Current enrollment

Public interface:

- Constructor with validation (code can't be empty, capacity must be positive)
- Getters for all fields
- `bool enrollStudent()` - returns false if at capacity
- `void dropStudent()` - decreases enrolled count
- `int getAvailableSeats() const`

Write a `main()` that creates a course, attempts enrollments, and demonstrates encapsulation.

Exercise E: Inheritance Chain

Create a three-level inheritance hierarchy:

1. **Person:** Base class with ``id``, ``name``
2. **Employee:** Inherits Person, adds ``salary``, ``department``
3. **Professor:** Inherits Employee, adds ``courses[]``, ``courseCount``

Requirements:

- Each constructor properly calls its parent constructor
- Each class has a `print()` method that reuses parent's `print()`
- Demonstrate construction/destruction order
- Show how `protected` allows derived class access

10. Suggested Answer Outlines

Exercise A Hints

- Global `X1` constructed before `main()`
- Each `func()` call: `X2` (local), `X3` (static - only once!), `X4`, `X5`
- Destruction: reverse of construction for automatic locals
- Static `X3` destroyed after `main()` ends

Exercise B Hints

Code block

```
1  class Professor {
2  private:
3      static int totalProfessors;
4  public:
5      Professor(...) { totalProfessors++; }
6      ~Professor() { totalProfessors--; }
7      static int getTotal() { return totalProfessors; }
8  };
9  int Professor::totalProfessors = 0;
```

Exercise C Key Points

- Array requires default constructor
- Use index tracking for "added" departments
- Bounds check before array access
- Print should handle "empty" slots

Exercise D Key Points

- All setters should validate
- `enrollStudent()` should check `enrolled < capacity`
- Negative counts should be prevented

- Demonstrate in main: try to over-enroll, verify it fails

Exercise E Key Points

Code block

```
1  class Person { /* id, name */ };
2  class Employee : public Person {
3      Employee(int id, string name, double sal)
4          : Person(id, name), salary(sal) {}
5  };
6  class Professor : public Employee {
7      Professor(int id, string name, double sal, string dept)
8          : Employee(id, name, sal), ... {}
9  };
```

11. Chapter Summary

Core Concepts

Variable Lifetime and Scope:

- **Local variables:** Automatic storage, created on scope entry, destroyed on exit
- **Global variables:** Static storage, created at program start, destroyed at program end
- **Static variables:** Static storage duration with local scope - persist between calls
- **Scope vs Lifetime:** Scope is where name is visible; lifetime is when object exists

Constructors and Initialization:

- Constructors establish the **beginning of an object's lifetime**
 - Initialization lists are preferred for efficiency and required for const/reference members
 - Default, parameterized, and copy constructors serve different purposes
 - Constructors should validate inputs to maintain object invariants

Destructors:

- Destructors mark the **end of an object's lifetime**
 - Called automatically when leaving scope or program end
- Run in **reverse order** of construction
 - Should not throw exceptions or contain complex logic

Object Arrays:

- Arrays of objects require default constructors
- All elements share the same lifetime as the array
- Elements destroyed in reverse index order

Access Control and Encapsulation:

- `private` : Only within class
- `protected` : Within class and derived classes
- `public` : Accessible everywhere
- Encapsulation hides internal state, exposes controlled interface

Inheritance:

- **Public inheritance**: Models "is-a" relationship
- **Private inheritance**: Models "implemented-in-terms-of"
 - Construction order: Base → Members → Derived
 - Destruction order: Derived → Members → Base (reverse)

Design Principles

1. **RAII**: Resource Acquisition Is Initialization — acquire resources in constructor, release in destructor
2. **Encapsulation**: Keep data private, expose behavior through public interfaces
3. **Prefer composition over inheritance** for "has-a" relationships
4. **Initialize everything**: Never leave members in undefined states
5. **Match lifetime to need**: Use locals for temporary data, statics for persistence, globals sparingly

Key Takeaways for withProf System

- Proper initialization prevents half-constructed objects
- Static members enable class-level tracking (total professors)
- Object arrays enable managing collections (faculty roster)
- Inheritance enables code reuse (Person → Professor)
- Access control protects invariants (private course list)
- Understanding lifetime prevents use-after-free bugs

End of Chapter 6: Constructors, Destructors, Inheritance, and Object Lifetime